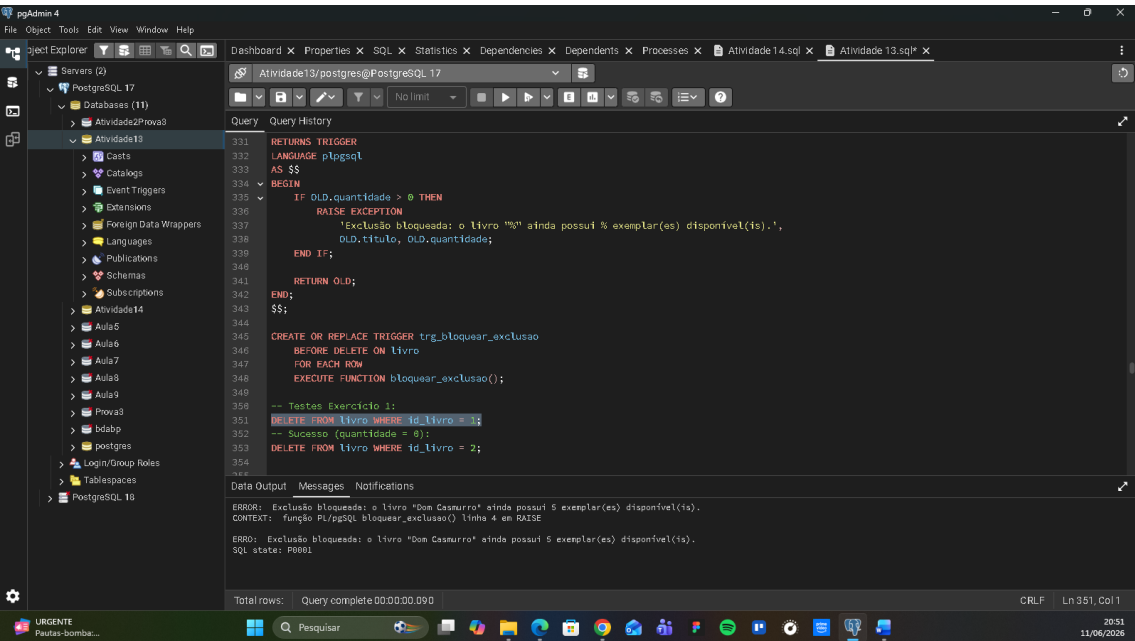
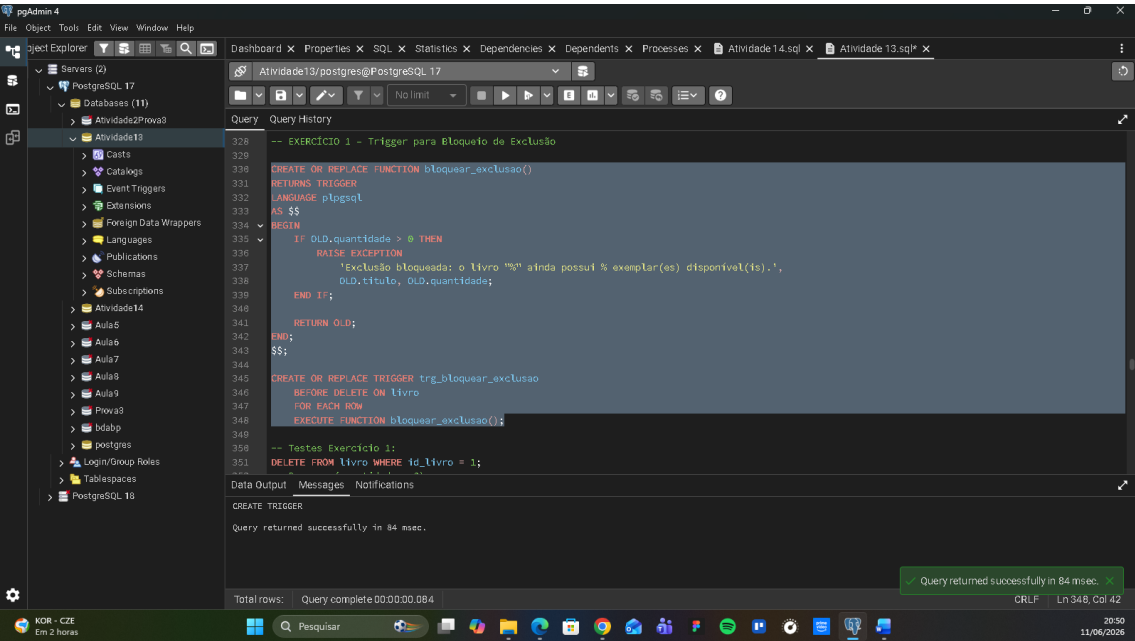
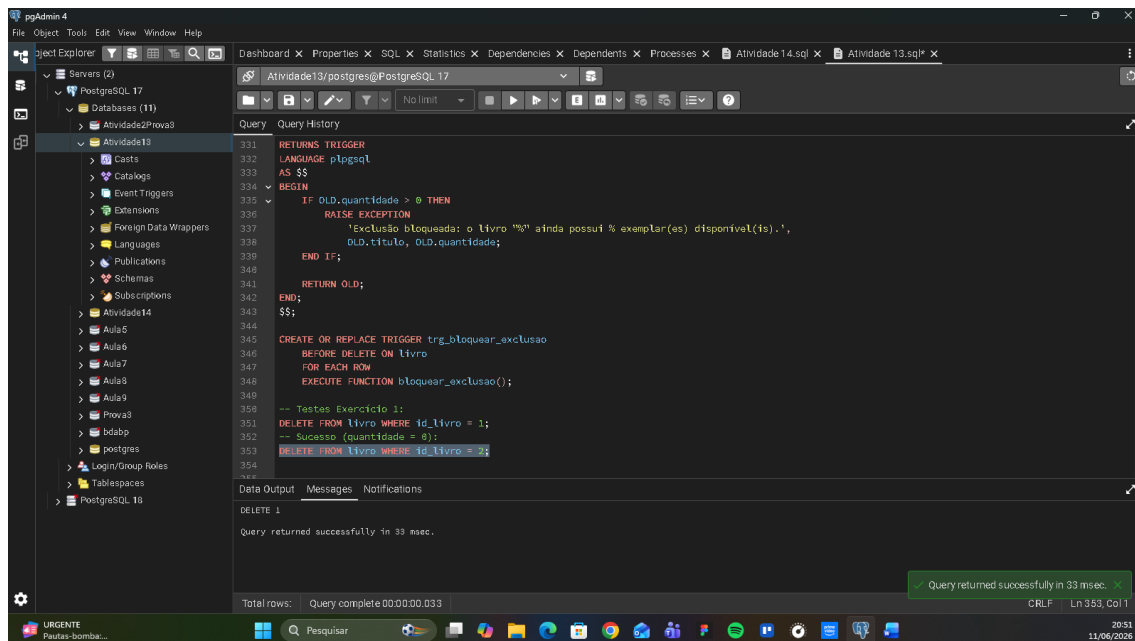


# Atividade Aula 16

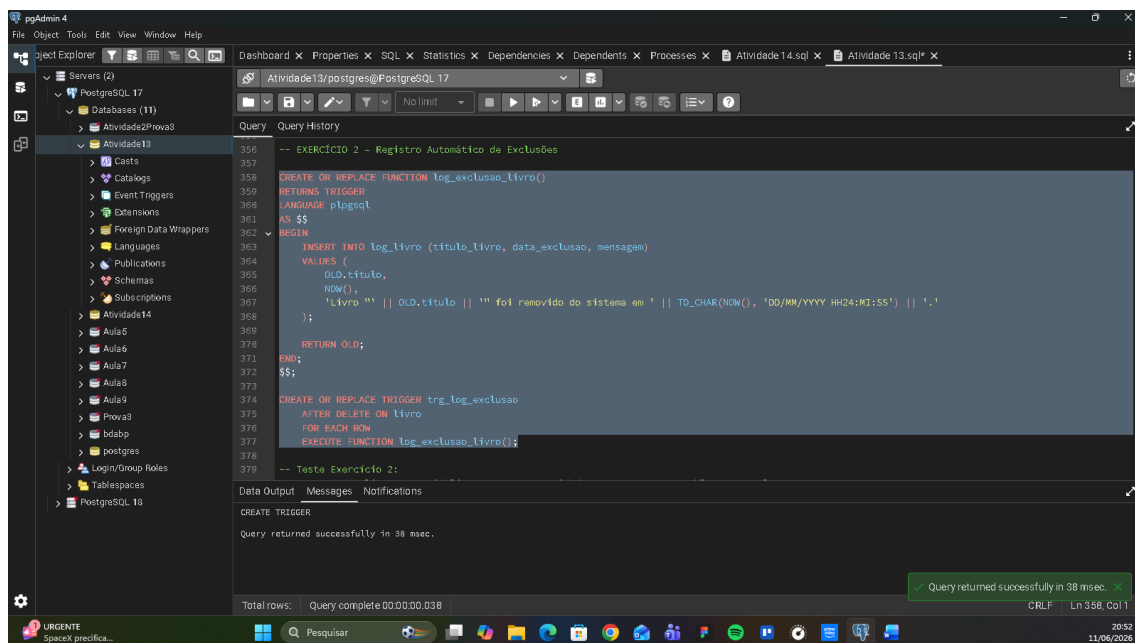
Luka Gomes

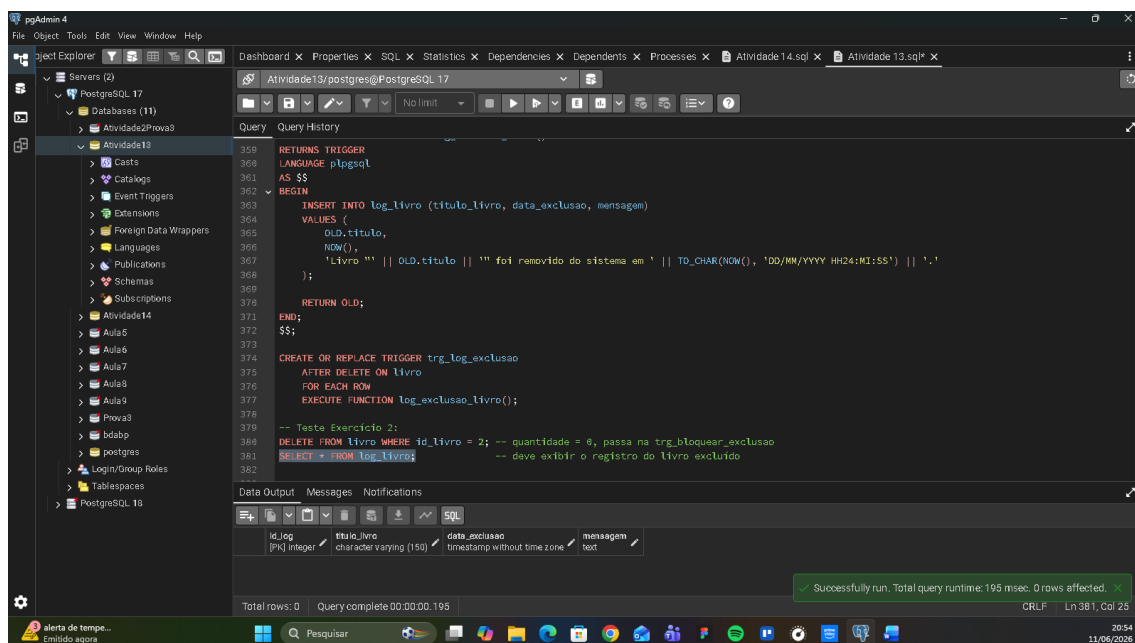
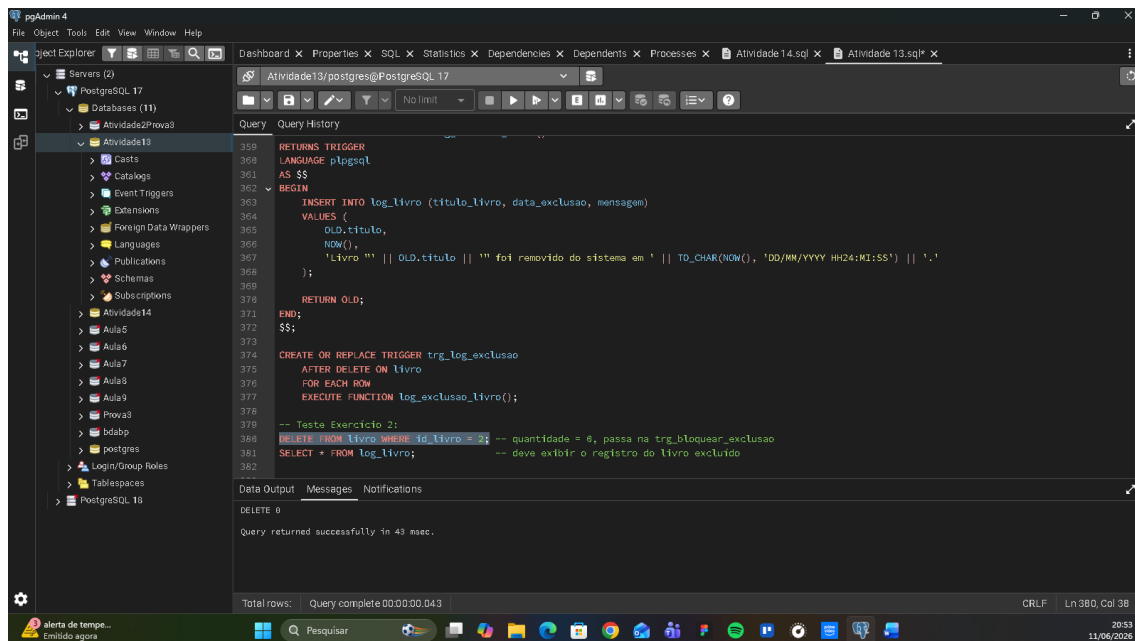
## Exercício 1





## Exercício 2





## Exercício 3

pgAdmin 4

File Object Tools Edit View Window Help

Servers (2)

- PostgreSQL 17
  - Databases (11)
    - Atividade2Prova3
      - Atividade18
        - Casts
        - Catalogs
        - Event Triggers
        - Extensions
        - Foreign Data Wrappers
        - Languages
        - Publications
        - Schemas
        - Subscriptions
        - Atividade14
        - Aula5
        - Aula6
        - Aula7
        - Aula8
        - Aula9
        - Prova3
        - bdabp
        - postgres
        - Login/Group Roles
        - Tablespaces
        - PostgreSQL 18

Dashboard Properties SQL Statistics Dependencies Dependents Processes Atividade14.sql Atividade13.sql

Query Query History

```
-- EXERCICIO 3 -- Controle de Aumento Excessivo de Estoque
384
385
386 CREATE OR REPLACE FUNCTION validar_limite_estoque()
387 RETURNS TRIGGER
388 LANGUAGE plpgsql
389 AS $$
390 BEGIN
391 IF NEW.quantidade > 100 THEN
392 RAISE EXCEPTION
393 'Atualização bloqueada: quantidade % excede o limite máximo permitido de 100 exemplares para o livro "%",',
394 NEW.quantidade, NEW.titulo;
395 END IF;
396 RETURN NEW;
397 END;
398 $$;
399
400 CREATE OR REPLACE TRIGGER trg_validar_limite
401 BEFORE UPDATE ON livro
402 FOR EACH ROW
403 EXECUTE FUNCTION validar_limite_estoque();
404
405 -- Testes Exercício 3:
406 -- Erro (quantidade > 100):
407 -- UPDATE livro SET quantidade = 150 WHERE id_livro = 1;
408
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 39 msec.

Total rows: Query complete 00:00:00.039 CRLF Ln 404, Col 47

20:54 11/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Servers (2)

- PostgreSQL 17
  - Databases (11)
    - Atividade2Prova3
      - Atividade18
        - Casts
        - Catalogs
        - Event Triggers
        - Extensions
        - Foreign Data Wrappers
        - Languages
        - Publications
        - Schemas
        - Subscriptions
        - Atividade14
        - Aula5
        - Aula6
        - Aula7
        - Aula8
        - Aula9
        - Prova3
        - bdabp
        - postgres
        - Login/Group Roles
        - Tablespaces
        - PostgreSQL 18

Dashboard Properties SQL Statistics Dependencies Dependents Processes Atividade14.sql Atividade13.sql

Query Query History

```
388 AS $$
389 BEGIN
390 IF NEW.quantidade > 100 THEN
391 RAISE EXCEPTION
392 'Atualização bloqueada: quantidade % excede o limite máximo permitido de 100 exemplares para o livro "%",',
393 NEW.quantidade, NEW.titulo;
394 END IF;
395 RETURN NEW;
396 END;
397 $$;
398
399 CREATE OR REPLACE TRIGGER trg_validar_limite
400 BEFORE UPDATE ON livro
401 FOR EACH ROW
402 EXECUTE FUNCTION validar_limite_estoque();
403
404 -- Testes Exercício 3:
405 -- Erro (quantidade > 100):
406 UPDATE livro SET quantidade = 150 WHERE id_livro = 1;
407 -- Sucesso:
408 UPDATE livro SET quantidade = 50 WHERE id_livro = 1;
409
410
411
412
413
```

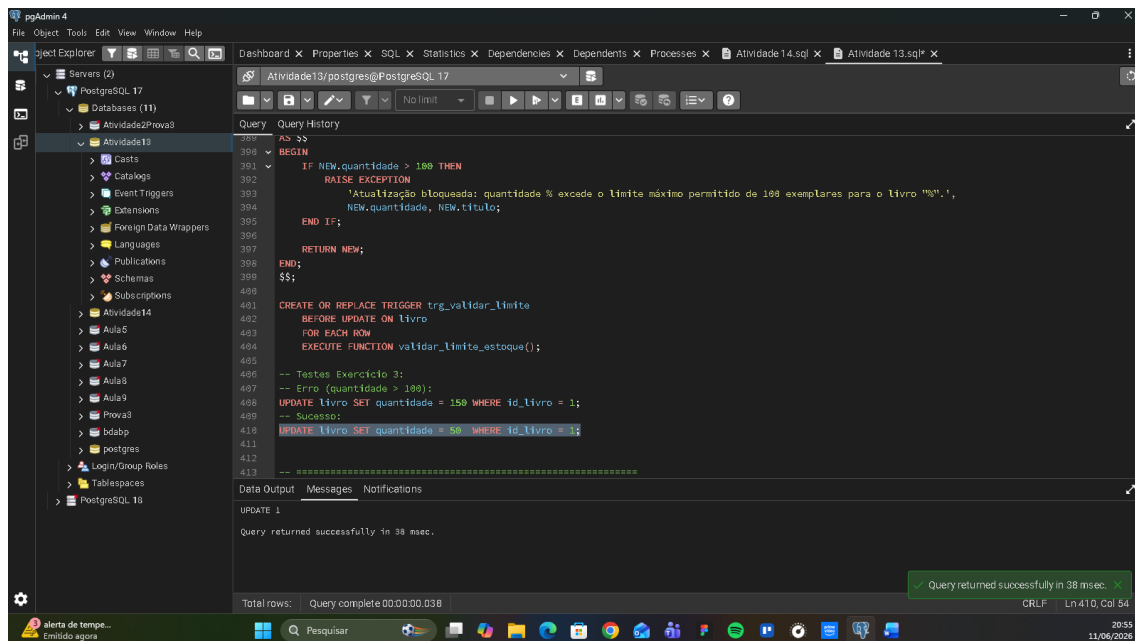
Data Output Messages Notifications

ERROR: Atualização bloqueada: quantidade 150 excede o limite máximo permitido de 100 exemplares para o livro "Dom Cassurro".  
CONTEXT: função PL/pgSQL validar\_limite\_estoque() linha 4 em RAISE

ERROR: Atualização bloqueada: quantidade 150 excede o limite máximo permitido de 100 exemplares para o livro "Dom Cassurro".  
SQL state: P0001

Total rows: Query complete 00:00:00.045 CRLF Ln 408, Col 1

20:55 11/06/2026



## Exercício 4

### a) DIFERENÇA ENTRE BEFORE E AFTER:

**BEFORE:** a trigger é executada ANTES da operação (INSERT, UPDATE ou DELETE) ser aplicada na tabela. Nesse momento ainda é possível cancelar a operação com RAISE EXCEPTION ou modificar os valores de NEW antes de gravá-los.

**AFTER:** a trigger é executada DEPOIS que a operação já foi concluída e confirmada na tabela. Os dados já estão persistidos, então não é possível cancelar a operação, mas é possível realizar ações com base no que foi alterado.

### b) QUAL USAR PARA VALIDAÇÃO:

**BEFORE** — porque ela intercepta a operação antes de acontecer, permitindo bloquear dados inválidos com RAISE EXCEPTION antes que qualquer alteração seja gravada no banco.

### c) QUAL USAR PARA AUDITORIA:

AFTER — porque nesse momento a operação já foi efetivada e os dados são consistentes. É o momento ideal para registrar logs, históricos e auditorias, pois sabe-se que a operação de fato ocorreu.

#### d) POR QUE A ORDEM DE EXECUÇÃO É IMPORTANTE:

Em bancos reais, várias triggers podem existir sobre a mesma tabela e evento.

A ordem importa porque:

- Uma trigger BEFORE de validação precisa rodar antes de qualquer AFTER de log, para que o log só seja gravado se a operação for válida.
- Se a ordem fosse invertida, poderíamos registrar um log de uma operação que depois seria bloqueada, gerando dados inconsistentes no histórico.
- Em operações encadeadas (trigger que dispara outra operação que aciona outra trigger), a ordem determina o estado do dado em cada etapa, podendo causar erros lógicos se não planejada corretamente.

#### Exercício 5

##### a) RISCOS DE REMOVER TRIGGERS E DEIXAR VALIDAÇÕES SÓ NA APLICAÇÃO:

- Qualquer acesso direto ao banco (scripts manuais, outras aplicações, imports) bypassará todas as validações, podendo inserir dados inconsistentes ou inválidos.
- Erros na aplicação ou falhas de deploy podem fazer com que versões sem validação cheguem a produção, corrompendo dados silenciosamente.
- Em ambientes com múltiplos sistemas acessando o mesmo banco (ex: ERP + portal web + app mobile), cada sistema precisaria replicar exatamente as mesmas regras, aumentando o risco de inconsistência entre elas.

#### b) VANTAGENS DE MANTER REGRAS DIRETAMENTE NO BANCO:

- As regras ficam centralizadas em um único lugar, independente de quantas aplicações acessam o banco.
- Garantem integridade mesmo em acessos diretos via SQL (pgAdmin, scripts, ETL).
- Reduzem duplicação de lógica entre diferentes sistemas ou camadas.
- São executadas de forma atômica junto à transação, sem risco de race conditions.

#### c) COMO TRIGGERS AJUDAM NA INTEGRIDADE E CONSISTÊNCIA:

Triggers automatizam regras de negócio diretamente no banco, garantindo que determinadas ações sempre ocorram em resposta a eventos, sem depender da aplicação. Por exemplo: ao deletar um livro, a trigger de log garante que o histórico SEMPRE será gravado; a trigger de bloqueio garante que exemplares disponíveis NUNCA serão removidos acidentalmente. Isso torna o banco autossuficiente na proteção de seus próprios dados.

#### d) EXEMPLO REAL EM SISTEMAS CORPORATIVOS:

Em sistemas de ERP (como SAP), triggers são usadas na movimentação de estoque: toda vez que um pedido de venda é confirmado (UPDATE no status do pedido), uma trigger AFTER UPDATE dispara automaticamente um débito na tabela de estoque do produto vendido. Isso garante que o estoque seja sempre atualizado de forma consistente, independente de qual módulo ou usuário gerou a venda.

Código SQL completo:

-- Atividade Aula 16

ALTER TABLE livro

ADD COLUMN IF NOT EXISTS quantidade INT NOT NULL DEFAULT 0;

-- Atualiza dados de exemplo com quantidades variadas

UPDATE livro SET quantidade = 5 WHERE id\_livro = 1;

UPDATE livro SET quantidade = 0 WHERE id\_livro = 2;

UPDATE livro SET quantidade = 3 WHERE id\_livro = 3;

UPDATE livro SET quantidade = 0 WHERE id\_livro = 4;

UPDATE livro SET quantidade = 2 WHERE id\_livro = 5;

UPDATE livro SET quantidade = 1 WHERE id\_livro = 6;

UPDATE livro SET quantidade = 0 WHERE id\_livro = 7;

-- EXERCÍCIO 2 – tabela de log

CREATE TABLE IF NOT EXISTS log\_livro (

id\_log SERIAL PRIMARY KEY,

titulo\_livro VARCHAR(150),

data\_exclusao TIMESTAMP DEFAULT NOW(),

mensagem TEXT

);

-- EXERCÍCIO 1 – Trigger para Bloqueio de Exclusão

CREATE OR REPLACE FUNCTION bloquear\_exclusao()

RETURNS TRIGGER

LANGUAGE plpgsql

AS \$\$



```
BEGIN

  IF OLD.quantidade > 0 THEN

    RAISE EXCEPTION

      'Exclusão bloqueada: o livro "%" ainda possui % exemplar(es) disponível(is).',

      OLD.titulo, OLD.quantidade;

  END IF;


  RETURN OLD;

END;

$$;
```

```
CREATE OR REPLACE TRIGGER trg_bloquear_exclusao

  BEFORE DELETE ON livro

  FOR EACH ROW

  EXECUTE FUNCTION bloquear_exclusao();
```

```
-- Testes Exercício 1:

DELETE FROM livro WHERE id_livro = 1;

-- Sucesso (quantidade = 0):

DELETE FROM livro WHERE id_livro = 2;
```

```
-- EXERCÍCIO 2 – Registro Automático de Exclusões
```

```
CREATE OR REPLACE FUNCTION log_exclusao_livro()

  RETURNS TRIGGER

  LANGUAGE plpgsql

  AS $$

  BEGIN

    INSERT INTO log_livro (titulo_livro, data_exclusao, mensagem)

    VALUES (
```

```

        OLD.titulo,

        NOW(),

        'Livro "' || OLD.titulo || '" foi removido do sistema em ' || TO_CHAR(NOW(),
        'DD/MM/YYYY HH24:MI:SS') || '

    );

    RETURN OLD;

END;

$$;

```

```

CREATE OR REPLACE TRIGGER trg_log_exclusao

    AFTER DELETE ON livro

    FOR EACH ROW

    EXECUTE FUNCTION log_exclusao_livro();

```

-- Teste Exercício 2:

```

DELETE FROM livro WHERE id_livro = 2; -- quantidade = 0, passa na trg_bloquear_exclusao

SELECT * FROM log_livro;           -- deve exibir o registro do livro excluído

```

-- EXERCÍCIO 3 – Controle de Aumento Excessivo de Estoque

```

CREATE OR REPLACE FUNCTION validar_limite_estoque()

    RETURNS TRIGGER

    LANGUAGE plpgsql

    AS $$

    BEGIN

        IF NEW.quantidade > 100 THEN

            RAISE EXCEPTION

                'Atualização bloqueada: quantidade % excede o limite máximo permitido de 100
                exemplares para o livro "%",

                NEW.quantidade, NEW.titulo;

```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$;
```

```
CREATE OR REPLACE TRIGGER trg_validar_limite
```

```
BEFORE UPDATE ON livro
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION validar_limite_estoque();
```

```
-- Testes Exercício 3:
```

```
-- Erro (quantidade > 100):
```

```
UPDATE livro SET quantidade = 150 WHERE id_livro = 1;
```

```
-- Sucesso:
```

```
UPDATE livro SET quantidade = 50 WHERE id_livro = 1;
```